A Comparative Study of Machine Learning and Tool-Augmented LLM Agents for Career Readiness and Skill-Gap Severity Prediction

Project Report

MSc Research Project
Artificial Intelligence

Srikaran Sankar
Student ID: 24201839

School of Computing
National College of Ireland

Supervisor: Prashanth Nayak

August 2026

National College of Ireland
Project Submission Sheet
School of Computing

Student Name:	Srikaran Sankar
Student ID:	24201839
Programme:	Artificial Intelligence
Year:	2026
Module:	MSc Research Project
Supervisor:	Prashanth Nayak
Submission Due Date:	10/08/2026
Project Title:	A Comparative Study of Machine Learning and Tool-Augmented LLM Agents for Career Readiness and Skill-Gap Severity Prediction
Word Count:	XXXX
Page Count:	22

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the bibliography section. Students are required to use the Referencing Standard specified in the report template. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action.

I agree to an electronic copy of my thesis being made publicly available on NORMA the National College of Ireland's Institutional Repository for consultation.

Signature:	
Date:	Wednesday 5th August, 2026

PLEASE READ THE FOLLOWING INSTRUCTIONS AND CHECKLIST:

Attach a completed copy of this sheet to each project (including multiple copies).	☐
Attach a Moodle submission receipt of the online project submission, to each project (including multiple copies).	☐
You must ensure that you retain a HARD COPY of the project, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer.	☐

Assignments that are submitted to the Programme Coordinator office must be placed into the assignment box located outside the office.

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

A Comparative Study of Machine Learning and Tool-Augmented LLM Agents for Career Readiness and Skill-Gap Severity Prediction

Srikaran Sankar
24201839

Abstract

A large gap between the skills of graduates and the skills needed by industry is a problem throughout higher education. The problem is treated here as a supervised prediction problem: given a candidate's skill profile, what is the difference between the skills they possess and the skills required for a job? Five solutions to this task are evaluated on a severity label, ranging from Low to High, derived from 4,416 technology job postings drawn from a public LinkedIn dataset. The target label is generated by a deterministic rule based on the number of taxonomy-skill matches in a posting (49 skill terms in total): postings are ranked by match count within each job category and split at the 33rd and 66th percentiles into Low, Medium and High severity. Because this label is a deterministic function of the skill count, and that count could easily leak into a model as a feature, the traditional classifiers are deliberately restricted to a generic TF-IDF representation of the raw posting text rather than the count itself, so that they must learn the underlying signal rather than recover the label-generating rule. On a held-out test set of 663 rows, the best discriminative model was Random Forest (accuracy: 72.4%, macro F1: 0.694), which outperformed SVM (accuracy: 67.0%, macro F1: 0.645) and DistilBERT (accuracy: 52.0%, macro F1: 0.441). A checklist-style LLM agent reached 68.0% accuracy (F1 = 0.667), while a deterministic-tool-using LLM agent reached 84.2% accuracy (F1 = 0.821) by relying on the same keyword taxonomy and matching function used to construct the ground truth. This research demonstrates that the tool-using agent's apparent success is attributable to being given access to the labelling mechanism itself, rather than to any superior reasoning about the job posting, and offers this as a methodological warning for the evaluation of tool-augmented agents against traditional classifiers. A working prototype of a Streamlit-based career-advisory application is also implemented and presented.

Keywords: career readiness, skill-gap analysis, random forest, SVM, DistilBERT, LLM agents, tool usage.

1 Introduction

1.1 Background and Motivation

Timescales for changing the training of new workers in the high-tech industry are more dynamic than those for updating higher-education programmes, and the imbalance between

the skills taught and the skills demanded has become a structural, rather than a temporary, characteristic of the labour market. The Future of Jobs Report (Di Battista et al.; 2023) by the World Economic Forum indicates that a significant share of the fundamental skills required by technology occupations is likely to shift over the next decade, with the result that graduates are left unemployed or underemployed while employers are simultaneously unable to fill positions that demand technical skills. This mismatch is not felt equally across the market: it is felt most acutely by early-career candidates who do not know which specific skills they lack.

University careers services try to close this gap through one-to-one advising, but manual gap analysis does not scale, the quality of advice varies from advisor to advisor, and it is not always grounded in current, evidence-based signals of what the market is actually demanding. This opens up the possibility of a data-driven solution: a candidate's CV can be reduced to a set of skills, compared against the skills required by a job posting or role at the time of deployment, and the extent of the difference between the two expressed as a supervised classification problem that a model can be trained to predict. Such a model could be integrated into a tool that gives a student an instant, evidence-based indicator of their readiness, together with a clear target path for closing the gap.

There are several distinct ways to approach this kind of prediction task, and they are not interchangeable. Ensemble methods such as Random Forest are effective on tabular data with heterogeneous features and provide explainable feature importances. Kernel methods such as an RBF-kernel SVM can learn non-linear decision boundaries. A transformer such as BERT, or its distilled variant DistilBERT, can in principle be trained directly on the natural-language text of a posting. A more recent alternative is the tool-augmented LLM agent, in which a language model solves a problem by delegating subtasks to external tools. These approaches trade off accuracy, interpretability, robustness and computational efficiency in different ways, and none is necessarily best suited to every problem. The purpose of this study is to compare which of these approaches is best suited to predicting skill-gap severity, and, more importantly, to understand *why*.

1.2 Problem Statement

Given a real collection of technology job postings, the problem addressed by this project is to:

- Develop a defensible, three-class skill-gap severity labelling procedure.
- Train and evaluate a representative sample of modelling techniques that predict that label.
- Determine which approach is most interpretable, most robust to imperfect input, and most useful in an actual educational career-guidance system.

A second, equally important question is: how much of any observed performance advantage , particularly one held by an LLM agent , can be attributed to genuine predictive ability, as opposed to an artefact of the experimental design?

1.3 Research Questions

RQ1: How do the accuracy and macro-averaged precision, recall and F1 of an ensemble method (Random Forest), a kernel-based method (SVM) and a fine-tuned transformer

(DistilBERT) compare on skill-gap severity classification, using a generic text representation of job postings?

RQ2: Can a tool-augmented LLM agent that makes its own semantic judgement of skill presence match a well-tuned traditional classifier on this task?

RQ3: How does the agent perform when it is permitted to delegate skill extraction to a deterministic, rule-replicating tool, and what does that level of performance actually measure?

1.4 Contributions

- A thorough comparison of Random Forest, SVM and DistilBERT on a generic TF-IDF representation of posting text, with deliberate omission of the skill-count feature that generates the label, so that the comparison is one of inference rather than look-up.
- An agentic extension that contrasts a reasoning-based checklist agent (RQ2) with a tool-delegating deterministic agent (RQ3), together with a discussion of what each result actually implies.
- A live career-advisory Streamlit application, built on the same taxonomy and benchmarks, that provides skill-gap analysis and a personalised action plan for non-technical users.
- A methodological contribution to agent evaluation: a concrete demonstration that an agent benchmark which reimplements the labelling oracle is measuring orchestration, not reasoning.
- A reproducible feature-engineering pipeline that transforms raw LinkedIn job descriptions into structured skill profiles using a taxonomy of 49 terms spanning technical skills, programming languages and soft skills.

The remainder of this report is structured as follows. Section 2 reviews the related literature on career-readiness prediction, skill taxonomies, and tool-augmented LLM agents. Section 3 describes the dataset, the target-construction procedure and the modelling methodology for all five approaches. Section 4 presents the architecture of the deployed career-advisory application. Section 5 and Section 6 present the results of the offline comparison, including confusion matrices and per-class analysis. Section 7 discusses the findings, their implications for agent evaluation, the limitations of the study, and directions for future work.

2 Related Work

Educational data mining (EDM) has grown into a sub-field concerned with predicting career readiness and employment outcomes from educational data. Castro-Lopez et al. (2022) used decision-tree and logistic-regression algorithms on academic-performance and extra-curricular data to predict graduate employment outcomes, concluding that these models were more predictive than grade-point average alone, and showing that employability is a multi-factorial construct that cannot be reduced to grades. Their feature space

neglected soft skills and professionally expressed intentions, however, which limits the actionability of their predictions: a model that predicts a poor outcome without indicating which skills are responsible provides little advisory value. Building on this, Alonso-Bello et al. (2020) used personality-survey instruments as features for employability classifiers, reporting improved F1 performance in multi-class outcome prediction, but limited to a single-cohort engineering sample.

These studies motivate two design commitments in the present work. First, the feature representation should include soft skills as well as technical skills, since soft skills also carry non-trivial predictive signal (Sancar and Cavus; 2025). Second, the output must be actionable and must identify specific missing skills rather than emit a bare severity level; the system described here therefore reports both a severity label and the specific missing skills.

2.1 Skill Taxonomies and Extraction

Levy and Murnane (2004) developed a theoretical basis for reasoning about skills in the labour market, building on the concept of skill complementarity and separating routine from non-routine cognitive and manual tasks. Most feature designs in career-prediction models, including the technical/soft-skill split used here, are still grounded in this taxonomy. Desai and Kulkarni (2025) tackled the operational challenge of measuring the gap directly, developing ontology-based taxonomies to map self-reported competencies to ESCO job profiles. They highlighted the difficulty of reconciling the idiosyncratic language of a CV or job advertisement with a fixed vocabulary as the main technical obstacle faced by any system attempting to match natural language against an online skills database. The present work adopts a pragmatic compromise to this problem, discussed critically in Section 3: a taxonomy of domain-specific terms matched case-insensitively against natural-language text, with word-boundary matching used to avoid spurious partial-token matches for short lexical items such as single-letter programming-language names.

Other approaches to skill extraction rely on supervised or distantly-supervised sequence labelling rather than lexical matching: Zhang et al. (2022) introduce the SkillSpan benchmark for span-level hard- and soft-skill extraction from job postings, Sayfullina et al. (2018) learn dense representations for soft-skill matching rather than relying on exact lexical overlap, and Decorte et al. (2022) address the label-noise problem inherent in distantly-supervised skill extraction through negative-sampling strategies. Boselli et al. (2018) instead classify whole job advertisements with standard machine-learning pipelines, closer in spirit to the posting-level classification task tackled in this study. The present work adopts the simpler lexical-matching approach over these more sophisticated extraction methods deliberately, since transparency and reproducibility of the ground-truth label construction (Section 3.3) is a stated design priority.

2.2 Classifiers and Transformers for Educational Prediction

The EDM survey by Romero and Ventura (2010) introduced the two main paradigms used to predict student performance , classification and clustering , and provides the methodological framework validating the classifier-based approach taken here. This study substantially reproduces the findings of Baciu et al. (2024) on student-dropout prediction, where an ensemble of machine-learning classifiers outperformed individual classifiers on tabular educational data, although by a smaller margin than reported in that work when

the classifiers are restricted to a generic text representation. On the transformer side, Sun et al. (2021) demonstrated the success of a fine-tuned BERT model on short, domain-specific text within a machine-reading-comprehension (MRC) framework, and Dobriţa et al. (2025) demonstrated an NLP-driven e-learning platform combining large language models with graph databases to deliver personalised educational guidance. The decision to include a transformer in the comparison, and to use DistilBERT (Sanh et al.; 2019) rather than the full BERT-base model (Devlin et al.; 2019), is justified by the fact that DistilBERT retains approximately 97% of BERT's language-understanding capability while using only 40% of its parameters and offering significantly faster inference , an important property for a system intended for real-time advising.

2.3 Tool-Augmented Language Agents

The agentic dimension of this study draws on two related lines of work. Yao et al. (2022) introduced the ReAct framework, in which a language model interleaves natural-language reasoning traces with tool use, allowing it to plan, execute, and reflect on and revise its actions. Because operations such as arithmetic, factual look-up and exact string matching are structurally vulnerable to language-model error, delegating them to external tools is an important pattern for tasks that language models are not well suited to perform themselves (Schick et al.; 2023). Both threads support a design principle underlying this work: an LLM should orchestrate, not compute; any step that requires exactness should be delegated to a deterministic tool. This pattern has since scaled substantially: Qin et al. (2023) demonstrate that LLMs can be taught to invoke over sixteen thousand real-world APIs, and Mialon et al. (2023) survey the broader landscape of augmented language models that combine reasoning with external tools, memory and retrieval. The present study's two agent variants sit at the simpler end of this spectrum , a handful of hand-written tools rather than a large API catalogue , which is precisely what makes it possible to audit exactly what each tool is permitted to know, the central methodological concern of Section 6. The deterministic-tool agent evaluated in Section 3 is motivated directly by this principle, as is the broader argument, developed at length in Section 6, that the agent's high empirical score must be interpreted with caution rather than taken at face value.

2.4 Data Leakage and Construct Validity

Kaufman et al. (2011) provide the standard definition of leakage in data mining: the inclusion, in the training data, of information about the target that would not legitimately be available at prediction time. They distinguish leakage in training examples from leakage in feature construction. Because the severity label used here is a deterministic function of a skill count within a job category, supplying that count as a feature would constitute feature-construction leakage; critically, such a leak would be invisible to conventional cross-validation, since the label is generated identically for every fold. This is why the traditional classifiers in this study are never given the skill count directly, and are instead restricted to a generic TF-IDF representation.

2.5 Research Gap

Although each individual strand of this literature is mature, no previous study systematically compares ensembles, kernel methods, transformers and tool-augmented LLM agents on a single, explicit skill-gap severity target while also accounting for the relationship between that target and what each model is permitted to see. Existing comparisons typically evaluate models with unequal access to information, or report accuracy on a constructed label without questioning whether the label is trivially separable from the available features. This report addresses that gap directly, and treats the resulting interrogation of *what each model is allowed to know* as more informative than a blunt accuracy comparison.

3 Dataset and Methodology

3.1 Dataset

The source data is a public LinkedIn job-postings dataset comprising roughly 124,000 scraped postings across all sectors. Postings were filtered to technology-related roles by a title-keyword match against a curated list of role titles, including "data scientist", "software engineer", "machine learning engineer", "data analyst", "cloud architect" and "cybersecurity analyst", yielding a working set of 4,416 postings. Each retained posting is mapped to one of seven job categories, such as Data Scientist / Machine Learning, Software Developer and Data Analyst. This mapping serves two purposes: it defines the peer group against which a posting's skill demand is judged, and it later provides the target-role selector in the deployed application.

Table 1: Overview of the dataset used for all experiments

Component	**Value**
Data Sources	`https://www.kaggle.com/datasets/arshkon/linkedin-job-postings`
Raw Postings	124,000
Technology filtered postings	4,416
Job Categories	7
Skill Taxonomy size	49 terms (19 technical, 16 programming languages, 14 soft skills)
Target classes	Low / Medium / High
Train / Validation / Test-Split	3,090 / 663 / 663 (70% / 15% / 15% stratified)
Random seed	42 (fixed throughout)
Held out test set size	663 postings

3.2 Skill Taxonomy and Feature Extraction

Every posting, and subsequently each CV, is analysed against a fixed taxonomy of 49 terms (19 technical skills, 16 programming languages and 14 soft skills). Matching against the raw text of a posting uses case-insensitive substring matching. Programming languages are matched using word boundaries rather than plain substring matching, to

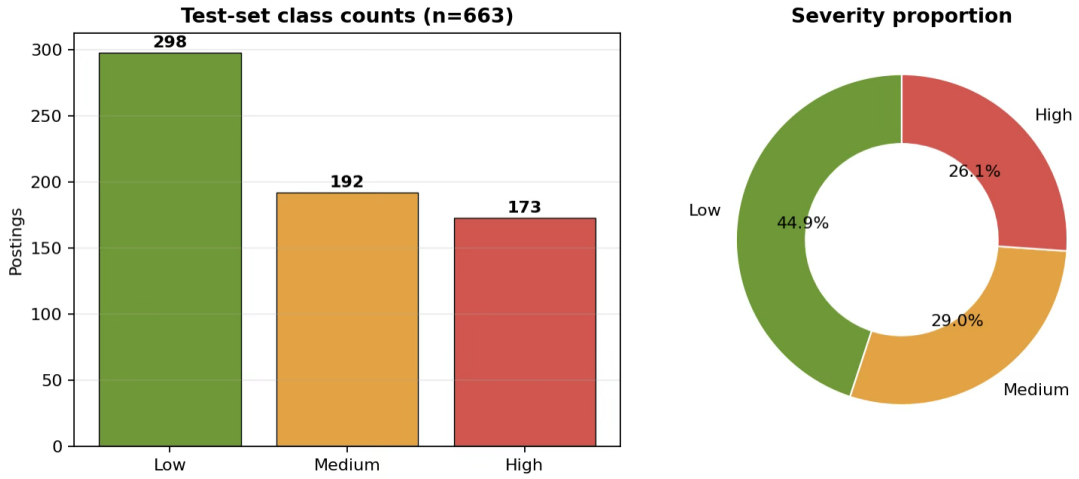

Figure 1: Distribution of three severity classes across the 663 row held out test set. Low is the majority class (298 postings) followed by Mediym (192 postings) and High (173 postings)

avoid partial-token false positives , for example, to prevent the single-letter language "R" from matching inside the common word "were". The number of unique matched terms for a posting is captured as `total_skills`, and the multi-hot skill vectors per category serve both to construct the label and, in the deployed system, to compute the skill gap.

3.3 Target Construction and Leakage Consideration

The skill-gap severity label is determined by how severe the skill gap is relative to percentile thresholds computed for each job category, based on `total_skills`. For each job category, the 33rd and 66th percentiles of `total_skills` are computed, and a posting is labelled Low if its skill count does not exceed the 33rd percentile for that category, Medium if it falls between the 33rd and 66th percentiles, and High if it exceeds the 66th percentile. Thresholds are computed per category, rather than globally, deliberately: if a data-engineering posting lists eleven skills and a business-analyst posting lists six, the business-analyst posting can still be judged more severe, because data-engineering postings tend to require more skills across the board.

This is a clear and reasonable operational definition, but it has a property that governs the entire experimental design and must be stated explicitly: the label is a deterministic function of `total_skills`, given a job category. If a model were given `total_skills` (or the category thresholds) as a feature, it would not be predicting severity , it would be recomputing a rule it had already been given. Following the leakage taxonomy of Kaufman et al. (2011), the three traditional classifiers in this study are therefore restricted to a generic TF-IDF representation of the raw posting text; `total_skills` itself is never supplied as a feature. This single design choice is the reason the reported accuracies fall in a realistic range, rather than the near-perfect scores that would result from a leaked feature.

The held-out test set is not significantly imbalanced, but Low accounts for 45% of test postings (298 postings), Medium 29% (192 postings) and High 26% (173 postings). This distribution has two implications. First, it establishes a naive majority-class baseline of 45% accuracy. Second, the comparatively smaller Medium and High classes foreshadow

a pattern seen throughout the results: relative to the well-populated Low class, the tail classes , and particularly the boundary between Low and Medium , are harder to model.

3.4 Traditional Classifiers on a Generic Text Representation

All three traditional classifiers use a TF-IDF (term frequency–inverse document frequency) vector representation of the posting text, limited to the 3,000 most frequent terms. TF-IDF weights each term by how often it appears in a posting, discounted by how often it appears across the corpus, so that terms distinctive to a posting are weighted more heavily. Crucially, this representation is indirect: the presence or absence of the token "python" in the description produces only a non-zero dimension, without telling the model how many skills are present in total. The model must learn the pattern of presence and absence of skill-related tokens that is associated with each severity level.

3.4.1 Random Forest

Random Forest combines many decision trees, each trained on a bootstrapped subset of the training data and a random subset of features at every split. It was chosen for its suitability to a sparse, high-dimensional lexical matrix: it is robust to the large number of irrelevant dimensions such a matrix inevitably contains, it combines weak individual lexical signals into a strong aggregate decision, and its Gini-based feature importances are interpretable. Stratified cross-validation was used to tune hyperparameters (number of trees, maximum depth, minimum samples per leaf), using macro F1 as the selection criterion.

3.4.2 Support Vector Machine (RBF Kernel)

The SVM is implemented in a one-vs-rest configuration over the sparse TF-IDF space, using a radial basis function (RBF) kernel to map the input into a high-dimensional space in which non-linear decision boundaries become linear. An RBF kernel was chosen because severity is based on a count of term presences that is inherently non-linear with respect to any single term, making a linear SVM unsuitable. The regularisation parameter C and the kernel coefficient γ were tuned by grid search, with probability estimates enabled to obtain probabilistic outputs.

3.4.3 DistilBERT

DistilBERT (Sanh et al.; 2019) is a six-layer distilled transformer encoder, fine-tuned end-to-end on raw posting text with a three-way classification head (dropout 0.1). Training used AdamW with a learning rate of 2×10^{-5} for 3 epochs, selecting the checkpoint with the best validation macro F1. Unlike the tabular models, DistilBERT is given only the tokenised text directly, and must learn both the 49 taxonomy terms and the counting-and-thresholding rule that defines the label, without any explicit supervision indicating which words are important.

3.5 Checklist Agent (RQ2)

The checklist agent is this study's genuine test of LLM reasoning. Rather than freely identifying the skills present in a posting , an open-ended task in which a skill can easily

be missed , the agent is asked to make an explicit present/absent judgement for each of the 49 taxonomy terms. Reframing recall as "exhaustive verification" makes the task more tractable and verifiable for a language model, and represents the fairest test of an agent making its own semantic judgements. The agent then applies the same percentile-threshold rule used elsewhere to produce a severity class from its own affirmative judgements. Calibration examples were included in the prompt in an attempt to bring the agent's notion of skill presence closer to the strict keyword match used to construct the ground truth.

3.6 Deterministic-Tool Agent (RQ3)

The deterministic-tool agent follows the orchestration-first pattern from the tool-use literature (Yao et al.; 2022; Schick et al.; 2023). It makes no judgement about skill presence itself. Instead, it is given two deterministic tools and instructed to use them: `get_category_thresholds`, which returns the 33rd and 66th percentile boundaries for a given job category, and `extract_skills_exact`, which applies exactly the algorithm used to construct the ground-truth label to the first 3,400 characters of the input, returning the matched skills and total count. The agent's role reduces to a fixed sequence of calls: call the threshold tool, call the extraction tool, compare the returned count against the returned thresholds, and record the result via a `submit_classification` call. The severity decision itself is made by the rule embedded in the tool output, not by the language model. The 3,400-character processing window is a bounded design decision whose effect on the agent's error profile is examined in Section 6.

3.7 Evaluation Protocol

All five models are trained on the same 3,090-row training split and evaluated on the same 663-row held-out test set, using accuracy and macro-averaged precision, recall and F1 across the three severity classes. Macro averaging weights every class equally regardless of its size, which is the appropriate choice given the imbalance shown in Section 3.3: a model that simply predicts the majority Low class would otherwise obtain an inflated score. AUC-ROC is additionally computed for the traditional classifiers, which output calibrated class probabilities, but not for the agents, which output only a discrete class decision. All results reflect a single stratified split, using a fixed random seed of 42; the tool's processing window is an experimental design choice rather than an a-priori constraint. Both agents use a single DeepSeek-class LLM backbone accessed through a function-calling interface.

4 Design Specification

In addition to the offline model comparison, this research is realised as an interactive career-advisory application. This section describes the end-to-end architecture of the deployed system; Section 5 then traces the running application screen by screen. The structure of the deployed system directly reflects the orchestration-first design validated in Section 3: the deployed system is an embodiment of that design, not a separate implementation of it.

4.1 End-to-End Architecture

A Streamlit front end is connected to a DeepSeek meta-agent that coordinates a deterministic LinkedIn-tools agent, with the offline validation study shown as a separate block beneath the live pipeline (Figure 2). A user uploads a CV to the web application and selects a target career from a list of options. The input-parsing stage reads the uploaded file into a text buffer, using `pdfplumber` for PDF files and `python-docx` for Word files, and maps the selected career to one of the seven in-house job categories.

The DeepSeek meta-agent (model `deepseek-v4-flash`, accessed through the TensorX API) then takes over and runs a tool-calling loop that mirrors the deterministic agent evaluated offline. The agent calls `get_category_thresholds` to obtain the 33rd/66th-percentile thresholds for the selected category, then calls `extract_skills_exact` to obtain the matched skills and total count for the first 3,400 characters of the CV, and finally calls `submit_classification` to record a severity label together with a confidence score and a natural-language justification. As Figure 2 makes clear, the Low/Medium/High decision is computed by a rule inside the tool, not by the language model: the LLM orchestrates, it does not judge.

The threshold and role-specific benchmark profiles that the tools refer to are compiled at application start-up and cached from the same 4,416 LinkedIn postings used in the offline study. The gap-computation stage then compares the candidate's extracted skills against the benchmark profile for their chosen role, and the advisory-output stage generates a severity meter, a matched-versus-missing skills breakdown, a five-step action plan, and course recommendations. The offline validation study is deliberately kept separate, both architecturally and in the presentation of results: the study evaluates all five approaches on the held-out test set, while the deployed application executes only the deterministic agent, so that the study's results are never confused with the live application's behaviour.

5 Implementation

This section traces the deployed Streamlit application described architecturally in Section 4, screen by screen, to describe the concrete outputs produced by the implementation.

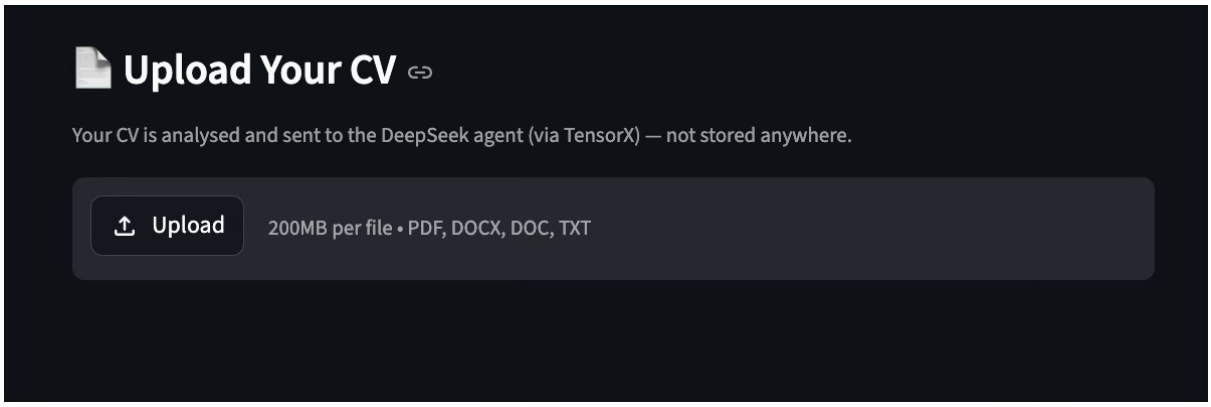

Figure 3: Landing Screen

The landing screen provides a file uploader for the candidate's CV, together with a sidebar in which the user selects a target career from the available options. On upload,

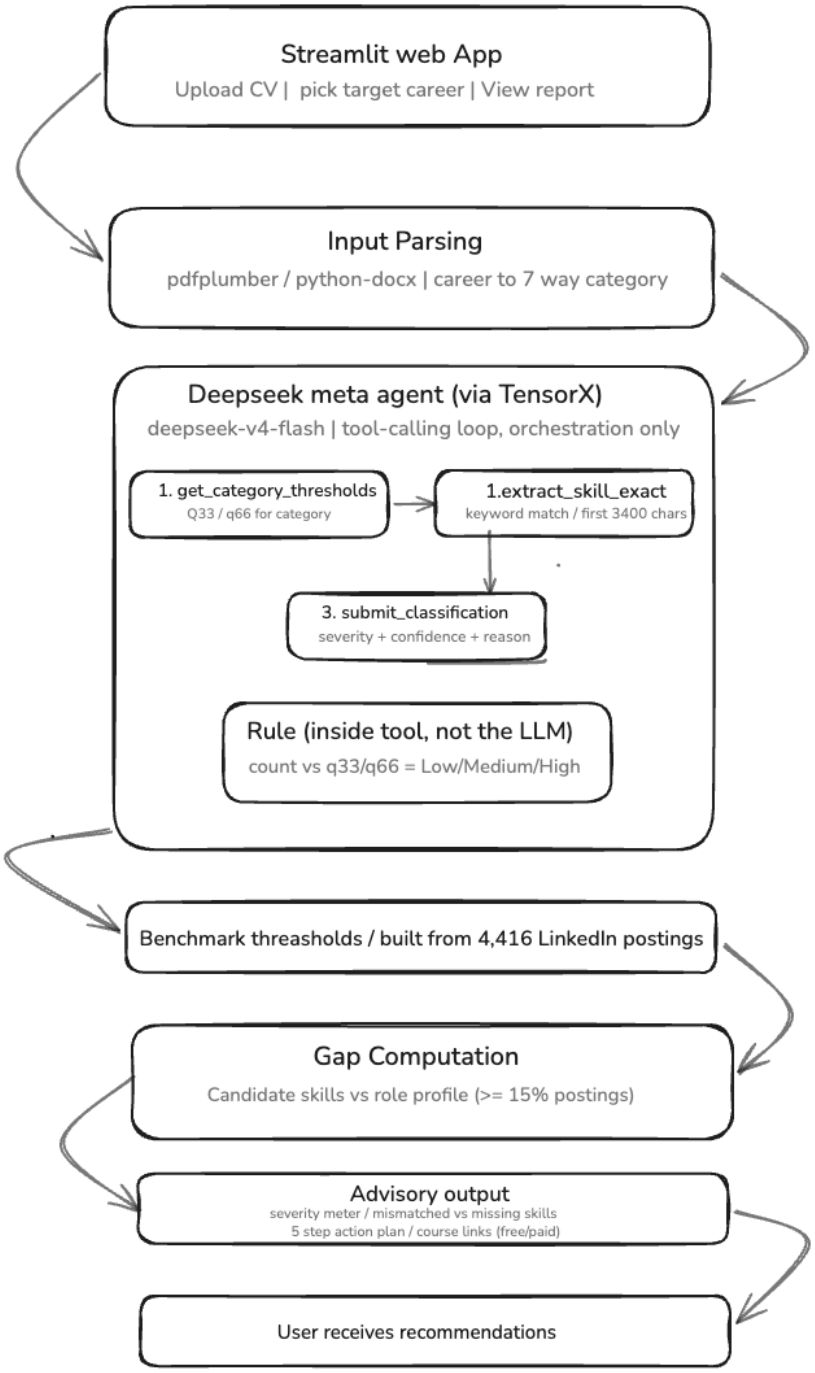

Figure 2: End-to-End Architecture of the deployed system

the application reads and formats the CV text for the agent. This screen corresponds to the Streamlit front-end and input-parsing stages of Figure 3.

Figure 4: Target Career

After a career is chosen, the application displays the benchmark profile built from LinkedIn postings matching that career: the full set of skills required for the role, and an average salary (where available) normalised across matching postings. This makes the standard against which the candidate is assessed visible to the user, rather than hidden inside the model.

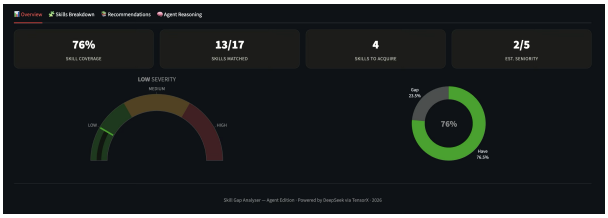

Figure 5: Result Dashboard

The results dashboard is the central component of the user experience. For a High result, the application returns the computed severity, a short natural-language summary (e.g. "significant gap, a focused learning plan is recommended"), and the confidence score returned by the agent. This screen implements the advisory-output stage of Figure 5, translating the study's classification outputs into a form usable by a non-technical user.

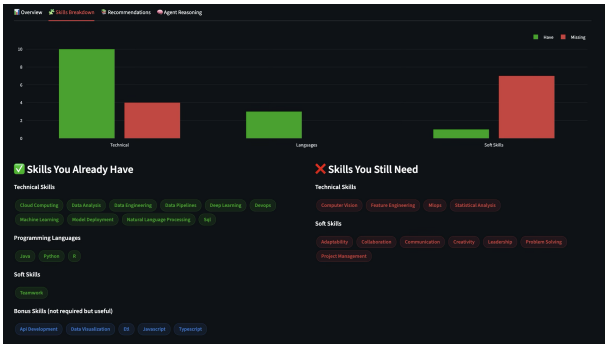

Figure 6: Skill Breakdown

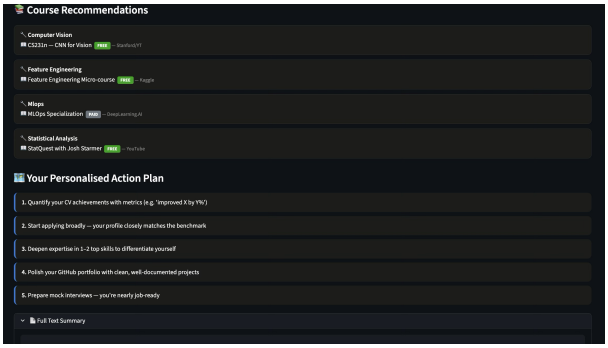

Figure 7: Course Recommendations

Below the verdict, the application breaks down the candidate's skill set into skills possessed and skills still required, computed as a set difference against the role benchmark. This directly addresses the gap identified in Castro-Lopez et al. (2022), that an abstract outcome prediction alone gives the user no actionable information: this screen names the specific skills to be learned.

Finally, the application generates a severity-aware, step-by-step action plan with an indicative timeline, alongside a curated set of free and paid learning resources for each missing skill. This completes the diagnostic-to-remediation cycle described in Section 4, delivering the full set of recommendations shown at the bottom of Figure 2.

6 Evaluation

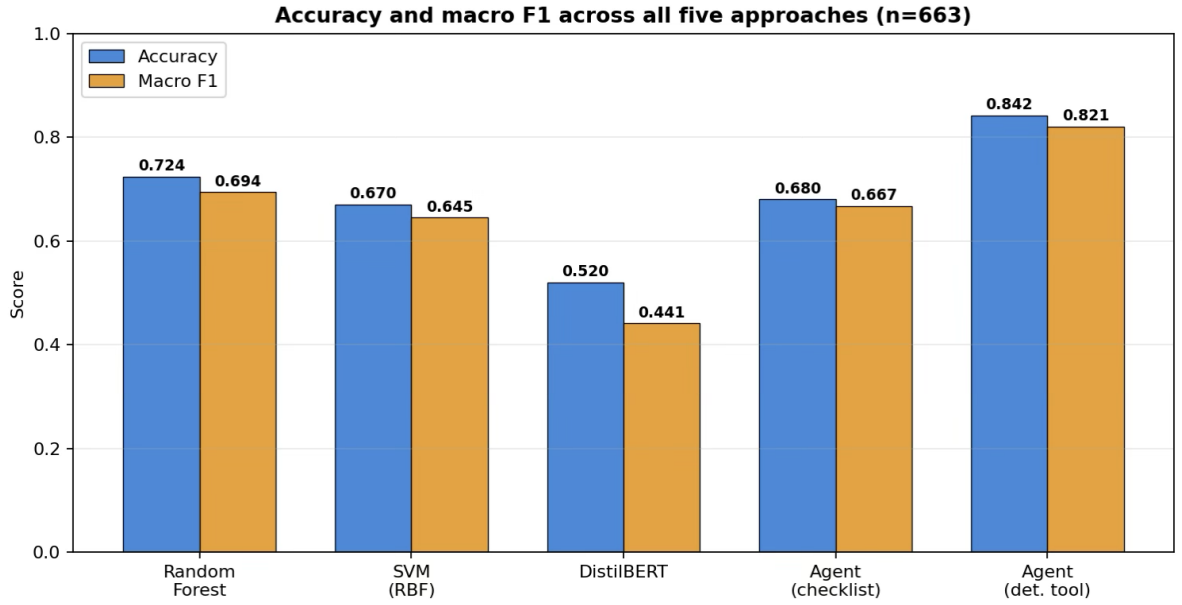

Figure 8: Accuracy and Macro F 1 for all given approaches

This section presents the complete quantitative results and interprets every chart and confusion matrix in turn. All figures are computed on the identical 663-row held-out test set.

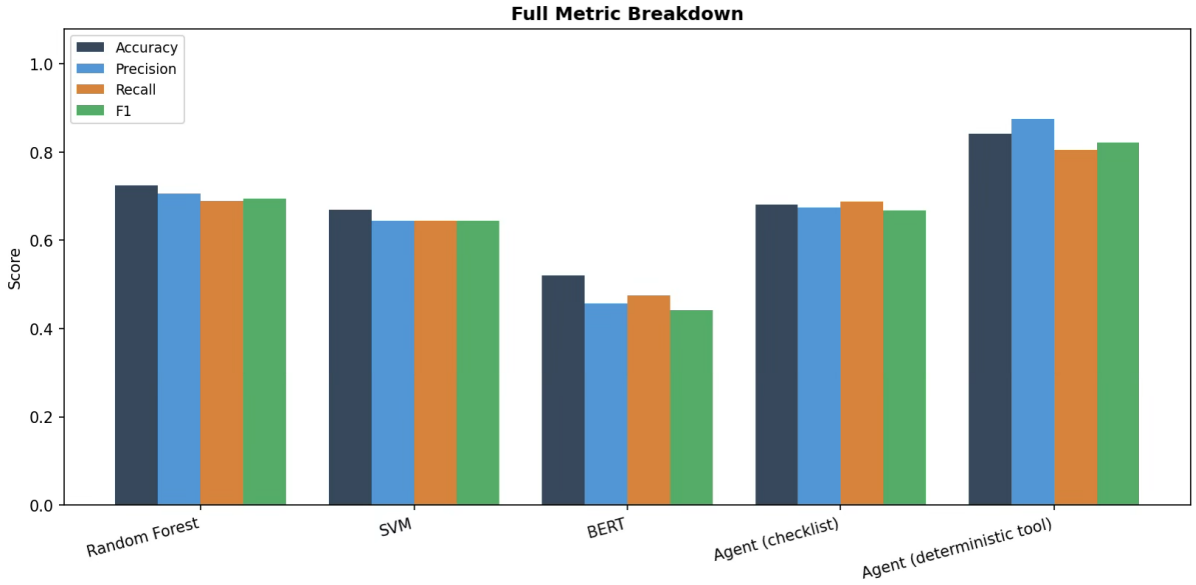

Figure 9: Full metrics breakdown as from analysis

6.1 Overall Comparison

Table 2: Held-out test performance for all five approaches (macro-averaged, $n = 663$).

Model	Accuracy	Precision	Recall	Macro F1
Random Forest	72.4%	0.706	0.689	0.694
SVM (RBF kernel)	67.0%	0.645	0.645	0.645
DistilBERT	52.0%	0.458	0.475	0.441
Checklist Agent	68.0%	0.674	0.688	0.667
Deterministic-Tool Agent	84.2%	0.876	0.805	0.821

Table 2 shows that, among the three discriminative models, Random Forest is the strongest, SVM is a consistent second, and DistilBERT is a consistent third. The two agents bracket this range at both extremes: the checklist agent sits slightly ahead of SVM and behind Random Forest, while the deterministic-tool agent is well ahead of every other model.

The most noticeable feature of the comparison is the difference between the two agent variants. Figure 8 shows accuracy and macro F1 side by side for all five approaches, and two features dominate. First, among the discriminative models the ordering is clear and simple: Random Forest, then SVM, then DistilBERT. Second, the two agent variants sit at opposite extremes of the whole spectrum of performance, even though they share the same base language model: the deterministic-tool agent outperforms every other model, while the checklist agent sits near the bottom. That two variants of the same LLM occupy such different positions is the visual crux of the study, and suggests that the deciding factor is not the intelligence of the LLM but what each variant is allowed to do. Macro F1 sits slightly below accuracy for every model, as expected given the class imbalance described in Section 3.3: errors on the smaller Medium and High classes pull down macro F1 more than they pull down raw accuracy.

The four metrics in Figure 9 sit compactly around 0.69–0.72 for Random Forest,

indicating good class balance , it neither over- nor under-predicts any class, a desirable property for an advisory system. SVM shows a similar balance at a slightly lower level, with precision, recall and F1 all around 0.645. For DistilBERT, accuracy (0.520) exceeds macro precision (0.458) and macro F1 (0.441), indicating that the model performs well on the majority class but poorly on at least one minority class, pulling down the macro average. The checklist agent's profile resembles Random Forest's, but with slightly less balance. The deterministic-tool agent is an outlier, with all four metrics above 0.80 and precision reaching 0.876, the highest single value recorded in the study.

6.2 Confusion Matrices

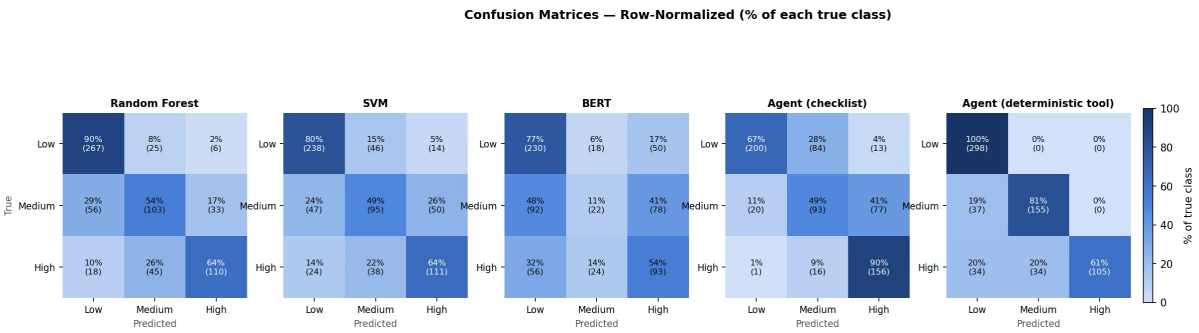

Figure 10: Row-normalised confusion matrices for all five approaches

Each cell of Figure 10 shows the percentage (and count) of a true class predicted as each class; an ideal model would show 100% on the diagonal. Rows are the actual severity classes and columns are the predicted classes, so the diagonal shows correct predictions and each off-diagonal cell shows a distinct type of error. The row-normalised view is used here, rather than the aggregate metrics above, because it gives a more diagnostic picture of where each model fails.

6.2.1 Random Forest

Random Forest classifies the Low class extremely well, at 90% (267 of 298 postings), and the High class solidly, at 64% (110 of 173). Its main weakness is the Medium class: only 54% (103 of 192) of true-Medium postings are recognised as such, with the remainder split between Low (29%) and High (17%). This is an intrinsic consequence of the target definition: Medium postings sit close to a decision boundary on both sides, and are therefore the most susceptible to misclassification. Because its errors spill into both neighbouring classes rather than being biased in one direction, the model shows low confidence near the boundary rather than a systematic directional bias.

6.2.2 SVM

SVM shows the same shape as Random Forest but at a uniformly lower strength: Low at 80% (238), Medium at 49% (95), High at 64% (111). The main difference from Random Forest is on the Low class, where SVM misroutes 15% to Medium and 5% to High, compared with 8% and 2% respectively for Random Forest. This is consistent with a kernel boundary being less able than an ensemble of trees to synthesise thousands of

sparse lexical features into a clean approximation of the underlying counting rule, making its Low-class predictions slightly more porous.

6.2.3 DistilBERT

DistilBERT's low overall score is explained by its confusion matrix. It performs well on the Low class (77%, 230 of 298) but performs close to chance on the Medium class, routing only 11% of true-Medium postings to Medium, with 48% misrouted to Low and 41% to High. Performance on High is moderate, at 54% (93 of 173). This suggests that learning the underlying task , recognising 49 specific terms and summing them , directly from raw text is genuinely difficult with only 3 epochs of fine-tuning on 3,090 training instances, without any explicit signal indicating which words carry the counting information.

6.2.4 Checklist Agent

The checklist agent's confusion matrix reveals an informative bias. It recovers the High class very well (90%, the best High recall of any model), but leaks the Low class upward: 28% of true-Low postings are predicted as High. Qualitative inspection suggests the agent extends easier credit for skill presence than a literal keyword match would, inferring skills from context even where the literal keyword is absent. This inflates its perceived skill counts and pushes borderline postings up a severity level, producing the observed upward bias from Low toward Medium and High. Notably, this bias persisted even after the prompt was supplemented with calibration examples, suggesting that an LLM's semantic judgement cannot easily be calibrated to match a strict keyword-matching criterion.

6.2.5 Deterministic-Tool Agent

The deterministic-tool agent's matrix is the most extreme, and the most revealing. The Low class is classified perfectly, at 100% (298 of 298), and the Medium class very well, at 81% (155 of 192), with the remainder misrouted to High. Its one significant weakness is the High class itself, at 61% (105 of 173), where 20% of true-High postings are under-classified as Low and 20% as Medium. This asymmetric error profile is a direct consequence of the tool's 3,400-character processing window: skills mentioned beyond that window are never counted, so a genuinely High posting can be under-rated. The key point is that every error made by this agent is a truncation error, unrelated to reasoning.

6.3 Per-Class F1

Figure 11 summarizes the confusion-matrix diagonals into per-class F1 scores, and a consistent cross-model pattern emerges: for every discriminative model the deepest trough sits on the Medium class and the shallowest on Low. The depth of the Medium trough tracks each model's dependence on exact counting: Random Forest loses around 0.28 F1 from Low (0.84) to Medium (0.56), while DistilBERT loses 0.51, falling from 0.68 to 0.17, effectively failing to learn the middle band at all. The deterministic-tool agent is the sole exception, scoring 0.81 on Medium, since its tool is exact and never needs to make a boundary judgment; its lowest bar instead falls on High (0.76), reflecting the truncation-driven under-counting discussed above.

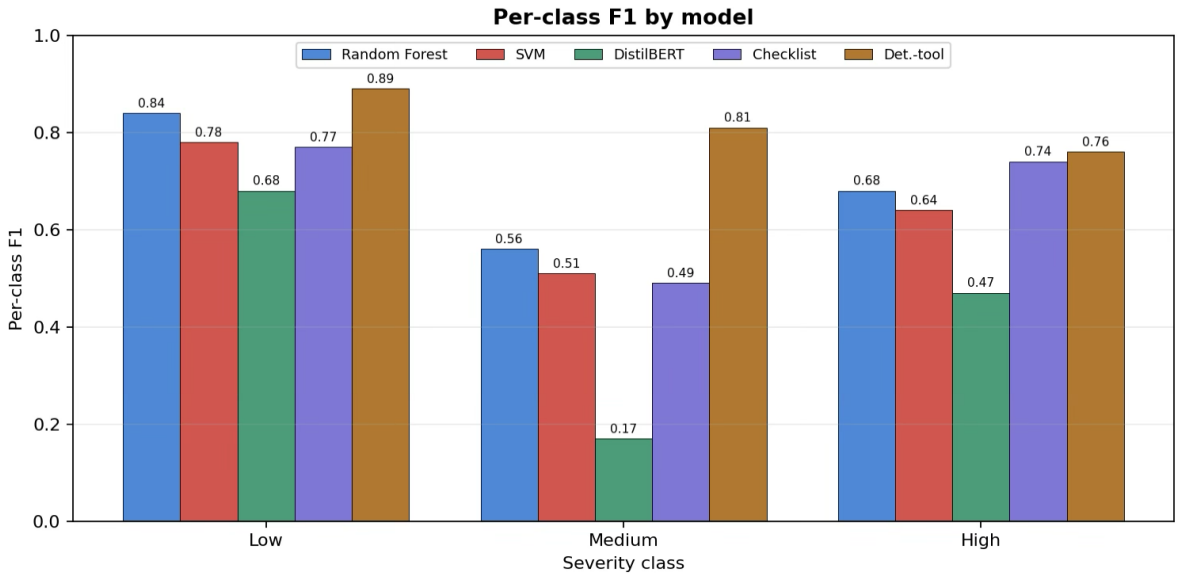

Figure 11: Per Class F1 for each model

6.4 Discussion

6.4.1 Answering the Research Questions

These results allow direct answers to each research question. For RQ1, the ordering of the discriminative models is clear: Random Forest (0.694 macro F1) outperforms SVM (0.645), which outperforms DistilBERT (0.441). The ensemble method is superior to the kernel method, and this gap roughly 20% smaller than reported by Baciu et al. (2024) on comparable tabular data is broadly consistent with their findings. DistilBERT's distant third place shows how difficult it is to learn an implicit counting-and-thresholding rule from limited text with only a modest amount of fine-tuning.

The answer to RQ2 is no. The checklist agent performs at a level comparable to SVM (0.667 macro F1) rather than exceeding it, and clearly below Random Forest. This study does not find that an LLM prompted by reasoning about skills outperforms a well-tuned classifier built on a generic text representation. For RQ3, the agent of the deterministic-tool achieves the highest macro F1 of the study (0.821), but, as argued in the following, this is attributable to what the agent is permitted to call rather than to any emergent reasoning capability.

6.4.2 What the Deterministic Agent's Score Actually Measures

The deterministic-tool agent is the top-performing model in this study, but it would be incorrect to conclude from this that agentic LLMs outperform classical machine learning at skill-gap prediction. This is the study's key methodological contribution, and the reasoning matters. The ground-truth label is constructed by running a keyword matcher, counting matched skills, and comparing that count against per-category percentile thresholds. The agent's `extract_skills_exact` tool *is* that same keyword matcher, and its `get_category_thresholds` tool returns those same thresholds. The agent is therefore given the label-construction procedure itself and asked to re-run it, which, on the large majority of postings, it does correctly.

It follows that the agent is not predicting in any meaningful sense , it is recomputing. Random Forest, by contrast, must infer the rule from lexical evidence alone, and reaches 72.4% accuracy doing so. The deterministic agent's 84.2% does not represent a better solution to the same problem; it represents a solution to a different, and much easier, problem: correctly orchestrating components that already contain the answer. The clearest evidence for this is the shape of its errors: every error it makes is a truncation error, occurring only when a skill lies outside the tool's 3,400-character window on a long posting. If the agent were given the whole posting, it would simply reproduce the labelling function exactly.

This generalises into a broader warning about agent evaluation. When the tools available to an agent are written by the same person who wrote the labelling function, it is easy , and often unintentional , to construct a benchmark on which the agent appears to reason well, when it is in fact calling the oracle. Evaluating a tool-augmented agent therefore requires scrutinising not just its score, but precisely what its tools are allowed to know. A benchmark built on a tool that reimplements the labelling oracle measures orchestration, not intelligence.

6.4.3 The Engineering Value of the Deterministic Design

None of this undermines the practical value of the deterministic design. It follows exactly the pattern recommended by the tool-use literature (Yao et al.; 2022; Schick et al.; 2023): build reliable, deterministic functions for exact string matching and exact numerical comparison, and leave the responsibility for deciding what to do with those results to the language model. This produces a system that is precise, that can incorporate a new tool without retraining, and that can explain its reasoning in natural language. These are genuine virtues for a day-to-day advisory application, but they are not the same thing as higher classification accuracy, and should not be mistaken for it.

6.4.4 The Checklist Agent's Over-Crediting

The checklist agent's characteristic error , over-crediting skills relative to the literal keyword standard , may reflect the agent being more human-like than the ground truth it is measured against, rather than less capable. A human reading "experience building predictive models" would reasonably infer machine-learning competence even without the literal keyword present; the strict matcher would not. The agent is effectively penalised for this kind of sensible inference. This is a limitation of the ground truth, not only of the agent, and motivates the recommendation (Section 7) to move toward a ground truth that better represents genuine skill rather than surface vocabulary.

6.4.5 Limitations and Threats to Validity

Proxy ground truth. The severity label is a synthetic keyword-count threshold rule, not a human-annotated judgement of skill-gap severity. Every model, including both agents, is measured against an approximation of the true construct rather than a strict ground truth. This is the most fundamental limitation of the study and constrains the interpretation of every reported score.

Single split and seed. All results reflect one stratified train/validation/test split with a fixed seed (42); no cross-validation or multi-seed variance estimate is reported.

Small differences , such as the gap between Random Forest and SVM , should therefore not be over-interpreted, as they may fall within the range of sampling variation.

Post-hoc design choices. The 3,000-term TF-IDF vocabulary and the tool's 3,400-character window were chosen to produce informative, non-saturated results, rather than derived from principled a-priori constraints, and their sensitivity has not been fully characterised.

Single LLM backbone. The agent results reflect one specific model (a DeepSeek-class model accessed via TensorX) and one prompt design. They may not generalise to other LLM backbones without re-validation, and the checklist agent's over-crediting bias in particular could differ across models.

Population shift. The models are trained on job postings but deployed against candidate CVs. This domain shift is not measured here, since no labelled CV data was available, and the vocabulary and structure of CVs may differ enough from postings to affect real-world accuracy.

Absence of a fairness audit. No demographic fairness analysis has been performed. Because the deployed application makes recommendations that affect people's career decisions, and automated resume-screening systems are known to encode bias (Cai et al.; 2024), such an audit is a prerequisite for any real deployment, not an optional extra.

7 Conclusion and Future Work

7.1 Conclusion

This dissertation compared Random Forest, SVM, DistilBERT and two tool-augmented LLM agent variants on the task of skill-gap severity classification, using a label derived from 4,416 LinkedIn technology postings and evaluating all five approaches on an identical 663-row held-out test set. Among the discriminative models, Random Forest was strongest (72.4% accuracy, 0.694 macro F1), ahead of SVM (67.0%, 0.645) and DistilBERT (52.0%, 0.441), with every model weakest on the boundary-defined Medium class. A checklist agent that reasoned about skills itself reached 68.0% and did not beat a well-tuned classifier, answering RQ2 in the negative. A deterministic-tool agent that delegated extraction to a rule-replicating function reached 84.2%, but this reflects the tool reproducing the labelling procedure rather than genuine semantic understanding, as confirmed by its truncation-only error profile.

The overarching conclusion is that current LLM agents do not out-reason a well-tuned classifier on an exact, rule-based classification task, and that apparent agent victories can be an artefact of tool design rather than evidence of reasoning. This is a useful negative result, and it carries a broader lesson for the evaluation of agentic systems: the question is never only how well an agent scored, but what its tools were allowed to know.

7.2 Future Work

The single most important next step is to replace the keyword-count proxy with a target that is exogenous to the features and closer to a genuine judgement of skill gap , for example, the distance between a candidate's skill profile and a role benchmark computed from a held-out slice of postings, with the raw skill count excluded from every model's inputs. This would make the label depend on a quantity the models do not directly

observe, would strip the deterministic agent of its shortcut, and would render the agent-versus-classifier comparison fully meaningful for the first time.

Beyond this, five further directions are planned:

- Replacing keyword matching with NER-based or embedding-based skill extraction, so that the ground truth tracks skills rather than vocabulary.
- Adding multi-seed variance estimates and cross-validation to establish the statistical significance of the classifier differences.
- Extending the agent evaluation to measure cost, latency and run-to-run consistency, none of which a discriminative classifier incurs.
- Extending the taxonomy multilingually via XLM-RoBERTa, to serve non-English job markets.
- Conducting a demographic fairness audit before any system built on this work is placed in front of a real student.

References

Alonso-Bello, E., Santana-Vega, L. E. and Feliciano-García, L. (2020). Employability skills of unaccompanied immigrant minors in canary islands, *Journal of New Approaches in Educational Research* **9**(1): 15–27.
URL: *https://doi.org/10.7821/naer.2020.1.433*

Baciu, V.-E., Stiens, J. and da Silva, B. (2024). MLinoBench: a comprehensive benchmarking tool for evaluating ML models on edge devices, *Journal of Systems Architecture* **155**: 103262.
URL: *https://doi.org/10.1016/j.sysarc.2024.103262*

Boselli, R., Cesarini, M., Mercorio, F. and Mezzanzanica, M. (2018). Classifying online job advertisements through machine learning, *Future Generation Computer Systems* **86**: 319–328.
URL: *https://doi.org/10.1016/j.future.2018.03.035*

Cai, F., Zhang, J. and Zhang, L. (2024). The impact of artificial intelligence replacing humans in making human resource management decisions on fairness: a case of resume screening, *Sustainability* **16**(9): 3840.
URL: *https://doi.org/10.3390/su16093840*

Castro-Lopez, A., Monteiro, S., Bernardo, A. B. and Almeida, L. S. (2022). Exploration of employability perceptions with blended multi-criteria decision-making methods, *Education + Training* **64**(2): 259–275.
URL: *https://doi.org/10.1108/ET-07-2021-0296*

Decorte, J.-J., Van Hautte, J., Deleu, J., Develder, C. and Demeester, T. (2022). Design of negative sampling strategies for distantly supervised skill extraction, arXiv preprint arXiv:2209.05987.
URL: *https://doi.org/10.48550/arXiv.2209.05987*

Desai, A. and Kulkarni, A. (2025). Unleashing the potential of ontology in skill extraction, *2025 1st International Conference on AIML-Applications for Engineering & Technology (ICAET)*, IEEE, pp. 1–6.
URL: *https://ieeexplore.ieee.org/document/10932270*

Devlin, J. et al. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186.
URL: *https://aclanthology.org/N19-1423/*

Di Battista, A. et al. (2023). Future of jobs report 2023, *Technical report*, World Economic Forum, Geneva.
URL: *https://www.weforum.org/reports/the-future-of-jobs-report-2023*

Dobriţa, G., Oprea, S.-V. and Bâra, A. (2025). An NLP-driven e-learning platform with LLMs and graph databases for personalised guidance, *Connection Science* **37**(1): 2518991.
URL: *https://doi.org/10.1080/09540091.2025.2518991*

Kaufman, S., Rosset, S. and Perlich, C. (2011). Leakage in data mining: formulation, detection, and avoidance, *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, New York, pp. 556–563.
URL: *https://doi.org/10.1145/2382577.2382579*

Levy, F. and Murnane, R. J. (2004). *The New Division of Labor: How Computers Are Creating the Next Job Market*, Princeton University Press, Princeton, NJ.
URL: *https://doi.org/10.1515/9781400845927*

Mialon, G., Dessì, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Raileanu, R., Rozière, B., Schick, T., Dwivedi-Yu, J., Celikyilmaz, A., Grave, E., LeCun, Y. and Scialom, T. (2023). Augmented language models: a survey, arXiv preprint arXiv:2302.07842.
URL: *https://arxiv.org/abs/2302.078422*

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z. and Sun, M. (2023). ToolLLM: facilitating large language models to master 16000+ real-world APIs, arXiv preprint arXiv:2307.16789.
URL: *https://doi.org/10.48550/arXiv.2307.16789*

Romero, C. and Ventura, S. (2010). Educational data mining: a review of the state of the art, *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* **40**(6): 601–618.
URL: *https://ieeexplore.ieee.org/document/5524021*

Sancar, N. and Cavus, N. (2025). Smart skills for smart cities: developing and validating an AI soft skills scale in the framework of the SDGs, *Sustainability* **17**(16): 7281.
URL: *https://doi.org/10.3390/su17167281.*

Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, arXiv preprint arXiv:1910.01108.
URL: *https://arxiv.org/abs/1910.01108*

Sayfullina, L., Malmi, E. and Kannala, J. (2018). Learning representations for soft skill matching, *International Conference on Analysis of Images, Social Networks and Texts*, Springer, pp. 141–152.
URL: *https://doi.org/10.1007/978-3-030-11027-7*$_1$4

Schick, T. et al. (2023). Toolformer: language models can teach themselves to use tools, *Advances in Neural Information Processing Systems* **36**: 68539–68551.
URL: *https://arxiv.org/abs/2302.04761*

Sun, C. et al. (2021). Biomedical named entity recognition using BERT in the machine reading comprehension framework, *Journal of Biomedical Informatics* **118**: 103799.
URL: *https://doi.org/10.1016/j.jbi.2021.103799*

Yao, S. et al. (2022). ReAct: Synergizing reasoning and acting in language models, *NeurIPS 2022 Foundation Models for Decision Making Workshop*.
URL: *https://arxiv.org/pdf/2210.03629*

Zhang, M., Jensen, K. N., Sonniks, S. and Plank, B. (2022). SkillSpan: hard and soft skill extraction from english job postings, *Proceedings of NAACL 2022*, pp. 4952–4967.
URL: *https://doi.org/10.18653/v1/2022.naacl-main.366*